SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

QUESTIONS: 05 **Total Marks:50**

Code of Conduct:

I Paleru.Dharani Govardhan / 192525280 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Paleru.Dharani Govardhan

Q1. Dataset: Online Shopping Transactions

Order ID Products Purchased

O1 Mobile, Earphones

O2 Mobile, Charger

O3 Earphones, Charger

O4 Mobile, Earphones, Charger

O5 Mobile, Earphones

Question:

Design pseudocode for the Apriori algorithm to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

Answer:

Given Dataset

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger

O5	Mobile, Earphones

Assume Minimum Support = 40%.

Since there are 5 transactions:

$$\text{Minimum Support Count} = 40\% \times 5 = 2$$

Therefore, an itemset must occur in at least 2 transactions to be frequent.

Pseudocode

Apriori(Transaction Database, Minimum Support)

1. Find all individual products and calculate their support.
2. Select products whose support >= Minimum Support.
   These form L1.
3. Set k = 2.
4. Repeat:
   a. Generate candidate itemsets Ck from L(k-1).
   b. Count the support of every candidate in Ck.
   c. Prune candidates whose support is less than
      Minimum Support.
   d. The remaining candidates form Lk.
   e. Increase k by 1.
5. Stop when no frequent itemsets are found.
6. Return all frequent itemsets.

Candidate Generation and Pruning

Iteration 1: 1-Itemsets

Item	Support Count	Support
Mobile	4	80%
Earphones	3	60%
Charger	3	60%

All three satisfy 40%, so:

L1 = {Mobile}, {Earphones}, {Charger}

Iteration 2: 2-Itemsets

Generate candidates:

- {Mobile, Earphones} → 3/5 = **60%** ✓
- {Mobile, Charger} → 2/5 = **40%** ✓
- {Earphones, Charger} → 2/5 = **40%** ✓

Therefore:

L2 = {Mobile, Earphones}, {Mobile, Charger}, {Earphones, Charger}

No candidate is pruned because all have support ≥ 40%.

Iteration 3: 3-Itemsets

Generate:

{Mobile, Earphones, Charger}

It occurs only in O4:

$$Support = \frac{1}{5} \times 100 = 20\%$$

Since 20% < 40%, it is pruned.

Therefore, there are no frequent 3-itemsets.

Final Frequent Itemsets

- **{Mobile}** → 80%
- **{Earphones}** → 60%
- **{Charger}** → 60%
- **{Mobile, Earphones}** → 60%
- **{Mobile, Charger}** → 40%
- **{Earphones, Charger}** → 40%

Conclusion: The Apriori algorithm generates candidate itemsets at each iteration and removes those that do not satisfy the minimum support. Here, the 3-itemset {Mobile, Earphones, Charger} is pruned because its support is only 20%.

Q2. Dataset: Library Book Borrowing Records

Borrow ID Books Borrowed

B1 Data Science, Python

B2 Python, Machine Learning

B3 Data Science, Machine Learning

B4 Data Science, Python, Machine Learning

B5 Python, Machine Learning

Question:

Develop pseudocode for the FP-Growth algorithm to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4)

Answer:

Given Dataset

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Assume Minimum Support = 40%.

Since there are 5 transactions:

$$\text{Minimum Support Count} = 40\% \times 5 = 2$$

So, an item must occur in at least 2 transactions to be frequent.

FP-Growth Pseudocode

FP-Growth(Database, Minimum Support)

1. Scan the database and count the frequency of each item.
2. Remove items whose support is below Minimum Support.
3. Arrange the remaining items in decreasing order of frequency.
4. Create the FP-Tree.
5. Insert each transaction into the FP-Tree according to
   the frequency order and update node counts.
6. Create a Header Table linking identical items.
7. Starting from the least frequent item, construct
   conditional pattern bases.
8. Build conditional FP-Trees and recursively mine them.
9. Return all frequent itemsets.

Step 1: Count Item Frequencies

Book	Support Count	Support
Python	4	80%
Machine Learning	4	80%
Data Science	3	60%

All three books are frequent.

Step 2: Arrange Items

In descending order of frequency:

Python → Machine Learning → Data Science

Step 3: Construct FP-Tree

Reorder each transaction according to the above order:

- B1: Python → Data Science
- B2: Python → Machine Learning
- B3: Machine Learning → Data Science
- B4: Python → Machine Learning → Data Science
- B5: Python → Machine Learning

The common prefixes are combined in the FP-Tree, reducing repeated storage.

A simplified FP-Tree is:

Root

├── Python: 4

│ ├── Machine Learning: 3

│ │ └── Data Science: 1

│ └── Data Science: 1

└── Machine Learning: 1

└── Data Science: 1

Step 4: Mine the FP-Tree

Frequent patterns include:

- **{Python}** → 4/5 = 80%
- **{Machine Learning}** → 4/5 = 80%
- **{Data Science}** → 3/5 = 60%
- **{Python, Machine Learning}** → 3/5 = 60%

- **{Python, Data Science}** → 2/5 = 40%
- **{Machine Learning, Data Science}** → 2/5 = 40%
- **{Python, Machine Learning, Data Science}** → 1/5 = 20% → Not frequent

Conclusion

FP-Growth first builds an FP-Tree by combining common transaction prefixes. It then mines the tree using conditional pattern bases and conditional FP-Trees. Unlike Apriori, FP-Growth does not generate candidate itemsets, which reduces the number of database scans and improves mining efficiency.

Q3. Dataset: Frequently Purchased Services

Service Combination Support Count

{Internet} 4

{Streaming} 4

{Cloud Storage} 3

{Internet, Streaming} 3

{Streaming, Cloud Storage} 2

Question:

Write pseudocode to generate association rules from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3)

Answer:

Given Frequent Service Combinations

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Assume **Minimum Confidence = 60%**.

Confidence Formula

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)} \times 100$$

Pseudocode

Generate-Rules(Frequent Itemsets, Minimum Confidence)

1. For each frequent itemset F:
2. Generate all possible non-empty subsets X of F.
3. Let Y = F - X.
4. Calculate confidence:

   Confidence = Support(F) / Support(X) × 100
5. If confidence >= Minimum Confidence:
6. Output rule X → Y
7. Else:
8. Discard the rule.
9. Return all valid association rules.

Rule Generation

From {Internet, Streaming}:

Rule 1: Internet → Streaming

$$Confidence = \frac{3}{4} \times 100 = 75\%$$

Since 75% ≥ 60%, it is **valid**.

Rule 2: Streaming → Internet

$$Confidence = \frac{3}{4} \times 100 = 75\%$$

Since 75% ≥ 60%, it is **valid**.

From {Streaming, Cloud Storage}:

Rule 3: Streaming → Cloud Storage

$$Confidence = \frac{2}{4} \times 100 = 50\%$$

50% < 60%, so it is discarded.

Rule 4: Cloud Storage → Streaming

$$Confidence = \frac{2}{3} \times 100 = 66.67\%$$

66.67% ≥ 60%, so it is valid.

Final Association Rules

Association Rule	Confidence	Result
Internet → Streaming	**75%**	Valid
Streaming → Internet	**75%**	Valid
Cloud Storage → Streaming	**66.67%**	Valid

Conclusion: The pseudocode generates possible rules from each frequent itemset, calculates their confidence, and filters out rules whose confidence is below the minimum threshold of 60%.

Q4. Dataset: Student Course Enrollment

Student ID Courses Enrolled

S1 Python, Database

S2 Python, AI

S3 Database, AI

S4 Python, Database, AI

S5 Python

Question:

Design pseudocode for constraint-based frequent itemset mining where only itemsets containing the course "Python"are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4)

Answer:

Given Dataset

Student ID	Courses Enrolled
S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Assume Minimum Support = 40%.

Since there are 5 students:

$$\text{Minimum Support Count} = 40\% \times 5 = 2$$

Pseudocode

Constraint-Based-Mining(Database, MinSupport)

1. Scan the database and find frequent items.

2. Keep only items that can form itemsets containing "Python".

3. Generate candidate itemsets only if they contain "Python".

4. Count the support of each candidate.

5. If support >= MinSupport:

Keep the itemset as frequent.

Else:

Prune the itemset.

6. Repeat until no more candidates can be generated.

7. Return the frequent itemsets containing "Python".

Candidate Generation and Pruning

Frequent itemsets containing Python are:

Itemset	Support Count	Support	Result
{Python}	4	80%	Frequent
{Python, Database}	2	40%	Frequent
{Python, AI}	2	40%	Frequent
{Python, Database, AI}	1	20%	Pruned

Effect of the Constraint

The constraint “itemset must contain Python” reduces the search space because itemsets that do not contain Python, such as {Database}, {AI}, and {Database, AI}, are not generated or considered.

This reduces the number of candidate itemsets and makes the mining process faster.

Conclusion

The frequent itemsets satisfying the Python constraint and 40% minimum support are:

{Python}, {Python, Database}, and {Python, AI}.

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID Treatments Received

P1 Healthcare, Cardiology, ECG

P2 Healthcare, Orthopedics

P3 Healthcare, Cardiology

P4 Healthcare, Cardiology, ECG

P5 Healthcare, Orthopedics

Question:

Develop pseudocode for multilevel association rule mining using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

Answer:

Given Dataset

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

The treatment hierarchy is:

Healthcare → Cardiology → ECG

and

Healthcare → Orthopedics

Assume Minimum Support = 40% and Minimum Confidence = 60%.

Pseudocode

Multilevel-Association-Mining(Database, Hierarchy)

1. Start from the highest level of the treatment hierarchy.

2. Count the support of treatments.

3. Generate frequent itemsets at this level.

4. Move to the next lower level of the hierarchy.

5. Generate candidate itemsets from frequent parent items.

6. Calculate support and keep frequent itemsets.

7. Generate association rules from the frequent itemsets.

8. Calculate confidence for each rule.

9. Keep rules whose confidence >= Minimum Confidence.

10. Repeat until the required hierarchy level is reached.

Rules at Different Levels

Level 1 – General level

Healthcare appears in all 5 patients:

$$Support(Healthcare) = \frac{5}{5} = 100\%$$

This gives a general pattern about healthcare treatment.

Level 2 – Specific treatment level

Cardiology appears in P1, P3 and P4:

$$Support(Cardiology) = \frac{3}{5} = 60\%$$

Orthopedics appears in P2 and P5:

$$Support(Orthopedics) = \frac{2}{5} = 40\%$$

Level 3 – Detailed level

ECG appears in P1 and P4:

$$Support(ECG) = \frac{2}{5} = 40\%$$

A meaningful rule is:

Cardiology → ECG

$$Confidence = \frac{Support(C, ardiologyECG)}{Support(Cardiology)} = \frac{2}{3} \times 100 = 66.67\%$$

Since 66.67% ≥ 60%, the rule is valid.

Role of Concept Hierarchy

A concept hierarchy allows patterns to be discovered from general to specific levels. At the higher level, we can identify broad patterns such as Healthcare. At lower levels, more specific patterns such as Cardiology → ECG can be discovered.

Thus, concept hierarchies improve pattern discovery by providing both general and detailed relationships and helping users understand treatment patterns at different levels of abstraction.